#AIR

HELLO - WORLD

Train a neural-network to recognize hand drawn numbers.

---

1. Problem and dataset

Mnist is a benchmark dataset of 70 000 grayscale handwritten digits (images 0 through 9).

This makes the dataset a multiclass problem, with 10-output classes.

Images provided by the mnist dataset are of size 28 X 28

Which means :

$$28 \cdot 28 = 784 \text{ pixels}$$

As the images are grayscale, by default eatch pixele has a value between 0 and 255. Where 255 meaning a completly lit white pixele and vice versa for a pixele with value 0.

To keep the data on a managable scale we normalize our inhut data often refered as X in ML :

$$X_{normalized} = \frac{X_{inputs}}{255}$$

This converts eatch pixel data from 0 -> 255 to a much more reasonable 0 —> 1.

Normalisation helps later on we for example calculating the weighted sum: $y = w \cdot X + b$

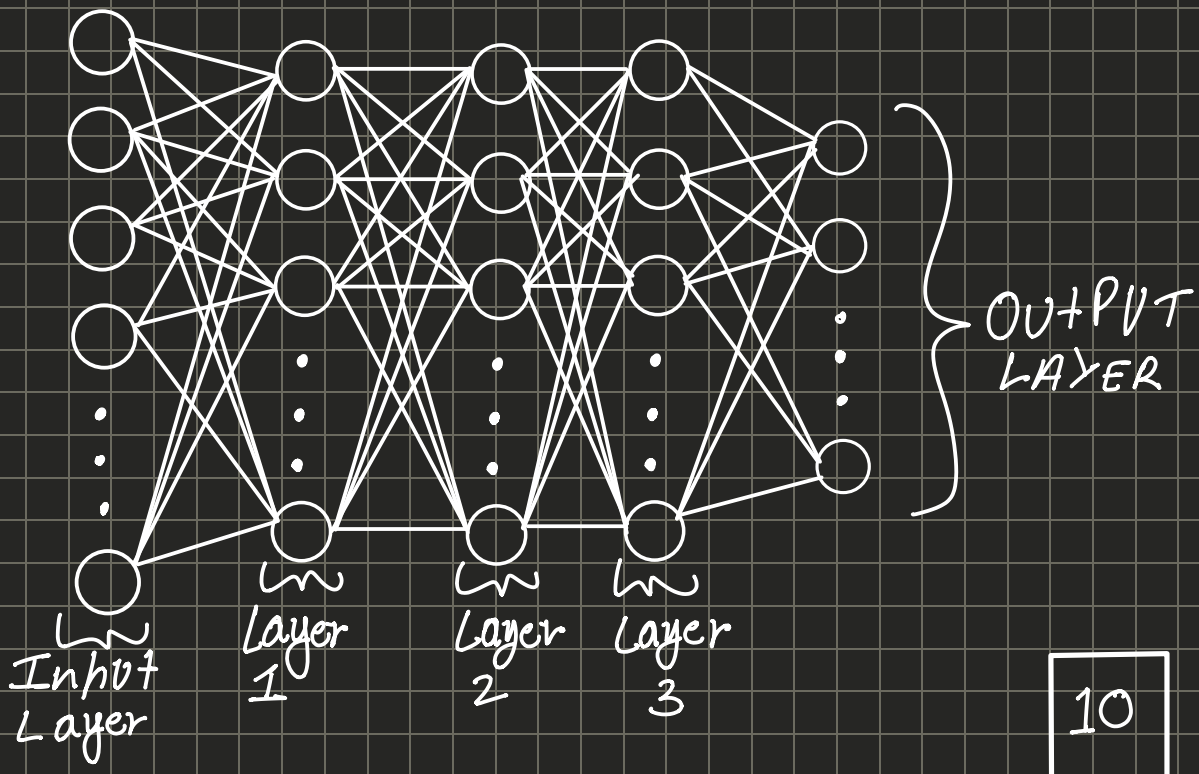
If X here was a big number like 255 our calculations would become unnecessarily large, causing complex gradients and etc... .

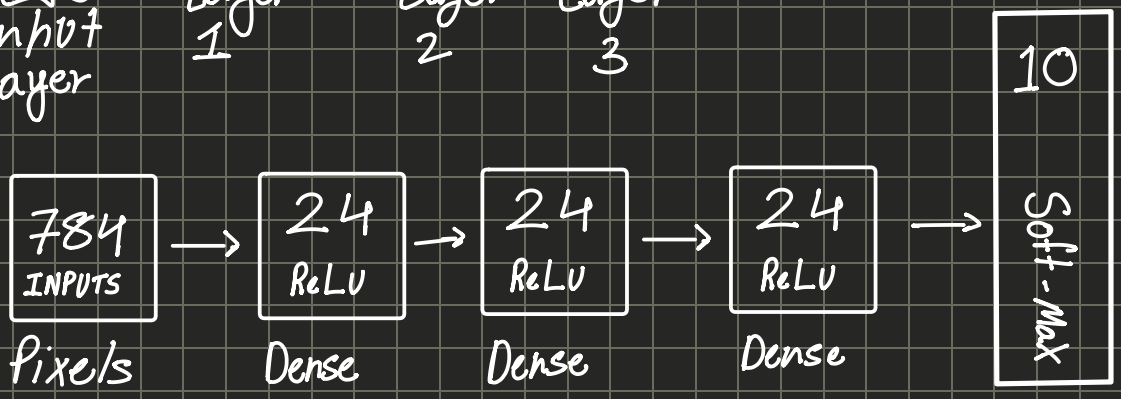
Note :

Teaching a Neural Network to recognize hand drawn digits is one of the very first things a beginner does in Deep learning, hence the project name Hello-World.

---

2. Architecture





Input layer :

The layer has 784 neurons and each one corresponding to a pixel of the image.

Layer 1 - Layer 3 : (Hiden Layers)

These layers are known as the hiden layer and eatch nueron on these layers are connected to all the nuersons on the previous layer.

These hiden layers are the reason a neural networke can learn. Eatch nueron are linear:

$$y = w \cdot x + b$$

Where w and b are the parameters we can play around with.

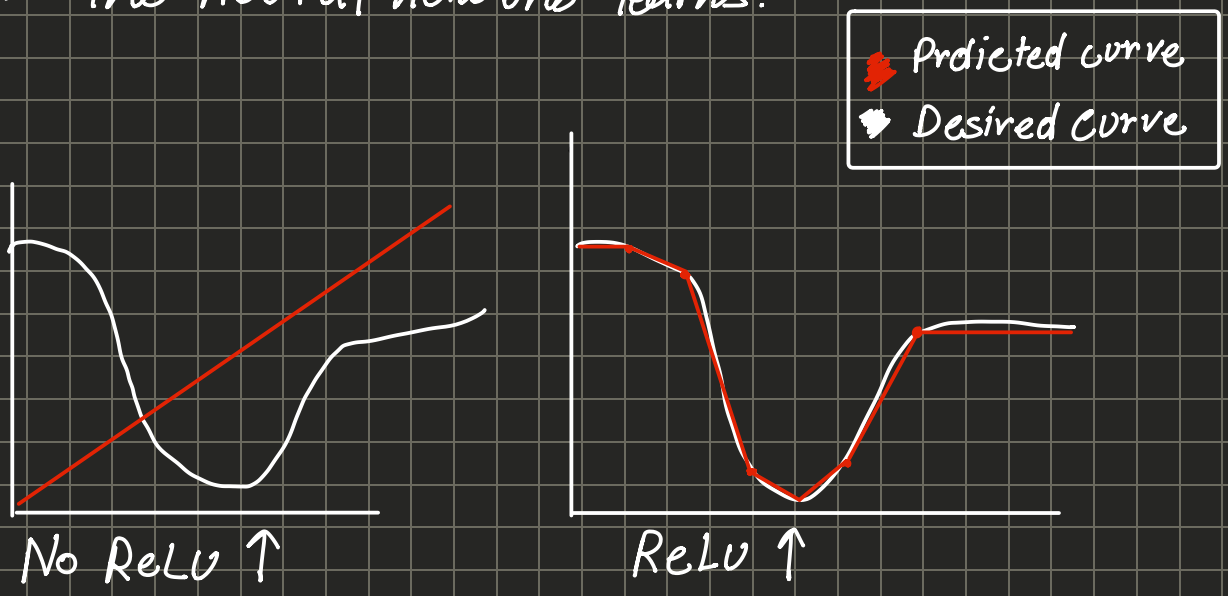
As these layer are linear learning/adating patterns can be hard and in most cases imposible. so we introduce some non-linearity with an activation function called ReLu.

ReLu:

Matamaticly ReLu is simple it changes any value bellow 0 to 0 so:

$$-1,27 \rightarrow \boxed{\text{ReLu}} \rightarrow 0$$

But consehtualy this has a huge effect on how the neural networke learns.



In the figure above imagine the white function is what we want to teach our model and in Red is what the model learns.

A linear NN. without ReLU can never match the desired function on the other hand ReLU makes it so the NN. can get close to matching the function.

Output Layer :

The output layer is a multi-classification layer that by default outputs a weightes sum of the previous layer.

We can change that by applying a Soft-Max activation. Soft-Max changes our outpus from just "random" weighted sum to probebilites that all add up to 1.

Logits

$$y \begin{bmatrix} 2,0 \\ 1,0 \\ 0,1 \end{bmatrix} \quad S(\hat{y}) = \frac{e^{\hat{y}}}{\sum_j e^{\hat{y}}}$$

Probabilites

$\longrightarrow P = 0,7$

$\longrightarrow P = 0,2$

$\longrightarrow P = 0,1$

In my code I dont directly apply Soft-Max as the cross entrophy loss function in Pytorch does that for me.

Loss function :

I used the cross entrophy loss function to calculate how wrong the Predictions are.

Cross entrophy loss :

$$L = -\sum_{i=1}^{c} y_i \log(P_i)$$

y_i = True values usualy a OHV

P_i = What the mode predicted

c = number of classes

Most of this just simplifys down to 0 so the final formula just becomes :

$$L = -\log(P_i)$$

Backprohogation:

This is one of the most important part of deep learning. We use the chain rule and gradient descent to minize the loss function.

Gradient descent:

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla J(\theta)$$

Where θ is a parameter (w or b)
η - Learning rate.

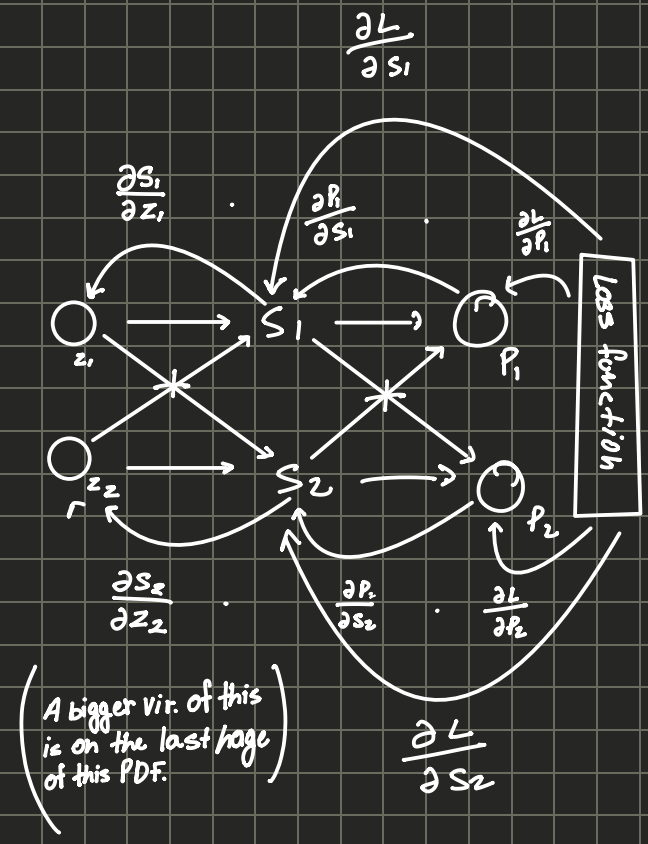
And later on we can update older params with new better values.

Chain Rule:

This is all about find out what parameter affects the lossfunction and by how much. If we know that we can use gradient descent to find values that make the Loss function as low as posible.

The lower the loss function the predictions the model makes.

How chain rule works.



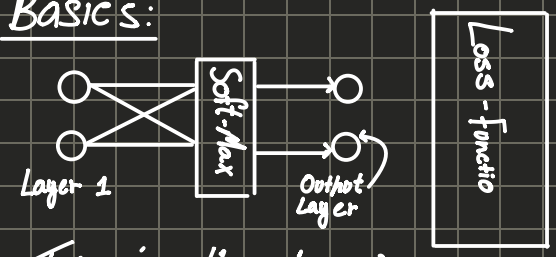
(A bigger vir. of this is on the last page of this PDF.)

Our job is to find how eatch param affectc the loss function for that we use derivatives.

$\frac{\partial L}{\partial \theta}$ => This says the partial derivative og L with respect to all the parameters.

I drew this figure to explain chain rule.

Basics:



This is the basic structure.

To go from the loss function L to the params of the output layer we simply do:

$\frac{\partial L}{\partial P_i}$ → This tells us how that P_i nueron is effecting the loss function. (see figure)

$Loss = -\log(P_i)$ soft-max

$$\frac{d}{dP_i}(\log P) = \frac{1}{P} \rightarrow \frac{\partial L}{\partial P_1} = P_i - y_i$$

We can't change P_i is a nueron, but we can change its weights and biases.

$$\left.\begin{aligned} \frac{\partial L}{\partial w_{P_i}} &= \frac{\partial L}{\partial P_i} \cdot \frac{\partial P}{\partial w_n} \\ \frac{\partial L}{\partial B_{P_i}} &= \frac{\partial L}{\partial P_i} \cdot \frac{\partial P}{\partial B} \end{aligned}\right\}$$ This tells us how the weights and baises of P_i effect the loss function.

To go even ferter back we use more chain rule

To get tom <u>Loss → S_i</u> (see figure)

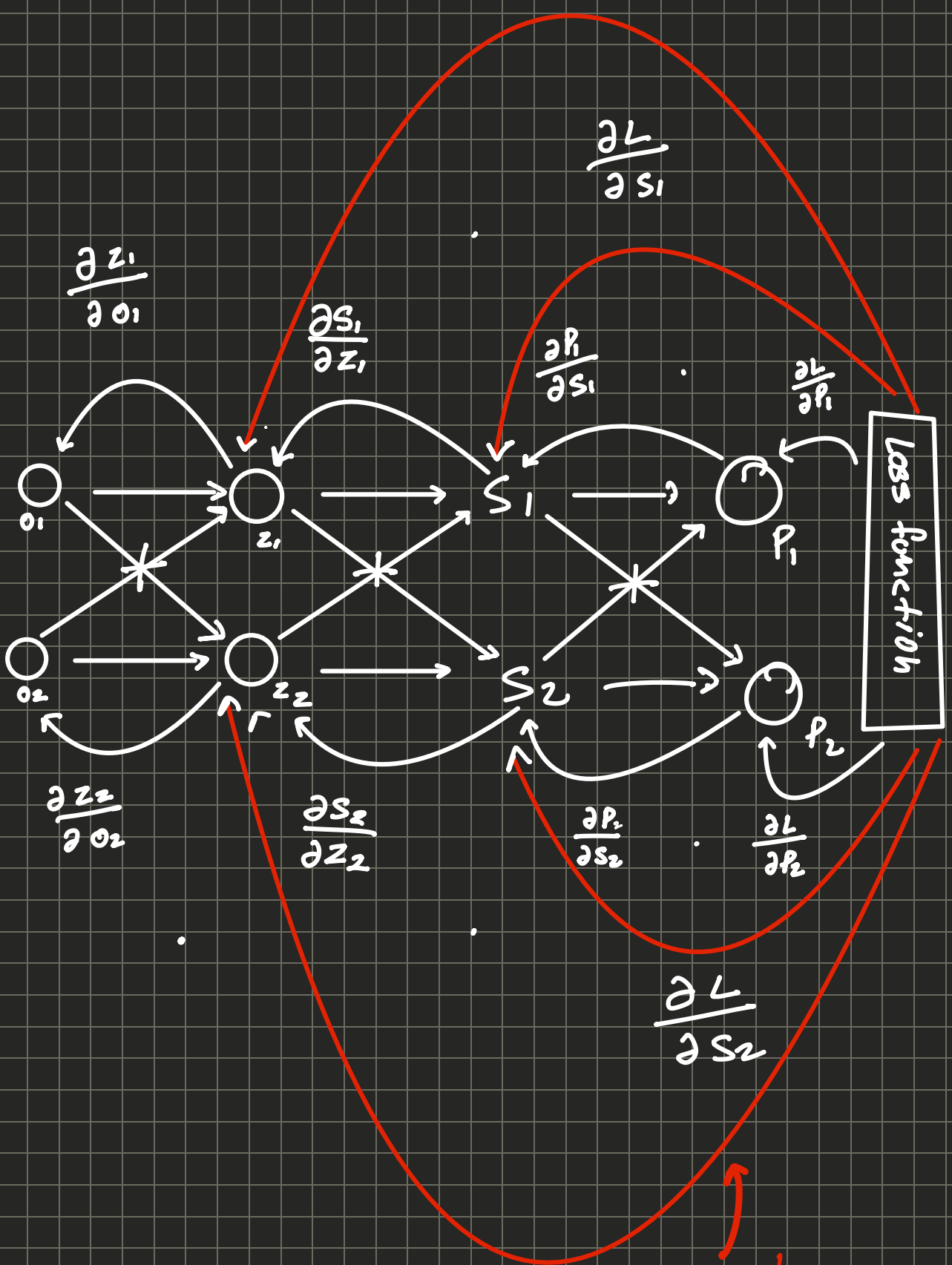
$$\frac{\partial L}{\partial S_1} = \frac{\partial L}{\partial P_1} \cdot \frac{\partial P_1}{\partial S_1} + \frac{\partial P_1}{\partial S_2} \cdot \frac{\partial L}{\partial S_2}$$

$$\frac{\partial L}{\partial S_2} = \frac{\partial L}{\partial P_2} \cdot \frac{\partial P_2}{\partial S_2} + \frac{\partial P_1}{\partial S_2} \cdot \frac{\partial L}{\partial S_1}$$

We need to sum because this is a dense neural-network an all nuerons in the previous layer affects a newron in this layer. So S_i effects both P_1 and P_2 see figure there is a big + in the middle of S and P layer.

You can go backward like this for as long as you want by using something I call abstractions we now know $\frac{\partial L}{\partial S_i}$ so everything before that doesn't mater to be able to go ferther back.

Chain Rule (in depth)



The red arrows are the abstractions.